

Developer's Manual

The code includes comments explaining the functionality/purpose of each function, as well as their input parameters (with their respective types) and return types.

Additionally, the following points are worth mentioning:

- **lexer.mli** This file contains all the possible tokens used in the parser. The **EOF** token has been redefined (now “;”) to support multiline inputs. The **eof** character is now **ignored**, along with **spaces** and the characters ‘\t’, ‘\n’, and ‘\r’.
Special attention should be given to the tokens **IDV** and **TYIDV**:
 - **IDV**
This token is used for the declaration of variables and functions in the code. It must **start** with a **lowercase letter**, followed by an alphanumeric combination of **numbers** and **lowercase letters**. It may also include the character “_”.
 - **TYIDV**
This token is used for the declaration of types in the code. It is useful for aliases, as well as for declaring records and variants. It must **start** with an **uppercase letter**, excluding the character “L”, which is reserved for **Lambda** (*short version*).
- **parser.mly** This file declares the grammar rules to be evaluated. It defines all the characters employed by the **lexer.mli** file. The main generation rules are **term**, **appTerm**, **atomicTerm**, **ty**, and **atomicTy**.
- **exceptions.mli** This file defines the exceptions used by the program. The main exceptions are **Type_error**, **Term_error**, and **NoRuleApplies**.
- **types.mli** This file defines all the types used by the interpreter. They are divided into two major groups: **types** and **terms**, represented as **ty** and **term** respectively. Both **ty** and **term** are further divided to simplify certain operations and reduce parentheses in the **string_of_term** and **string_of_ty** functions. These groups are:
 - **term** -> **atomicTerm**
 - **ty** -> **atomicTy**
- **termops.ml** and **termops.mli** These files contain the definitions of functions operating on terms.
- **typeops.ml** and **typeops.mli** These files contain the definitions of functions operating on types. They also include the “**typeof**” function (even though it applies to terms, it returns a type).
- **utils.ml** and **utils.mli** These files contain auxiliary functions used in the other files.
- **main.ml** This file hosts the main loop of the program.
- **examples.txt** Contains execution examples, which are also documented (more comprehensively) in the user manual.
- **Makefile** The “-g” parameter has been added to all compilation operations to

enable debugging mode.

Additionally, a **run** rule has been added, which will launch the interpreter with “**ledit**” and the parameter **OCAMLRUNPARAM=b** to display the stack trace in case an unhandled exception is raised.